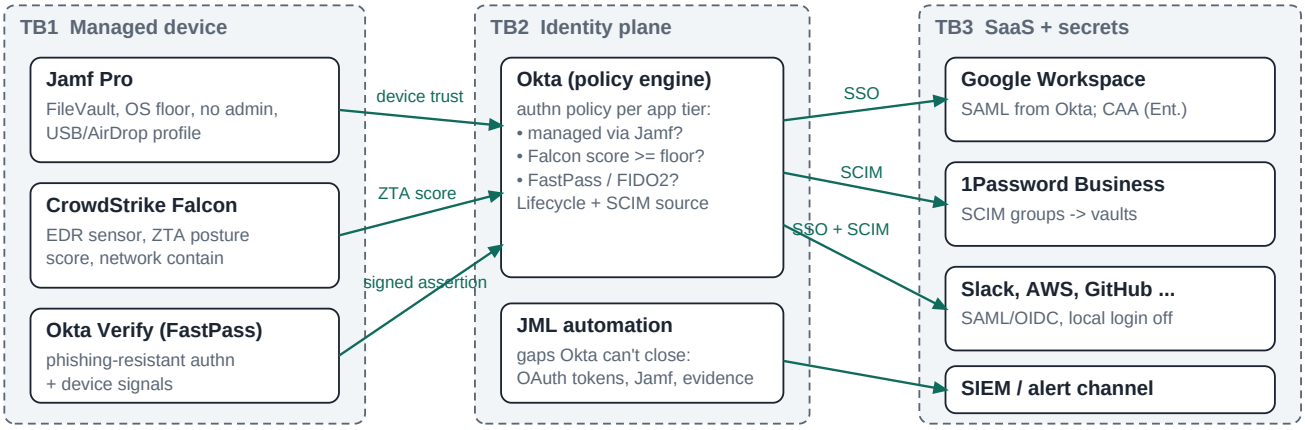
Mal — Endpoint-to-Identity Security Architecture

Companion to Part 1 (`mal-jml`). Vishnudev Ammuppilly · November 2026 · Assumptions: ~50 staff scaling to 200, macOS-majority fleet, Google Workspace as productivity suite, regulated fintech handling customer financial data.

1. Stack design rationale

Principle: Okta is the only door. Every SaaS app authenticates through Okta (SAML/OIDC) with local/password login disabled, so one policy engine decides access and one lifecycle event revokes it. Device posture is an *input* to that decision, never a parallel gate that users can route around.



EDR choice: CrowdStrike Falcon. Defender's main advantage is bundling with Microsoft 365 and Entra conditional access; Mal is on Google Workspace and Okta, so that advantage mostly disappears, and Falcon's macOS sensor and native Okta integration (Zero Trust Assessment score consumed in Okta device assurance) are stronger fits. Check Point Harmony is credible but has the weakest identity integration of the three. Cost is the honest counter-argument; at 50 seats it is material.

Trust boundaries

Boundary	What crosses it	Control
TB1 → TB2 device to IdP	User authentication + device signals	Okta Verify FastPass (phishing-resistant, bound to device keychain); Jamf device trust proves "managed"; Falcon ZTA score proves "healthy". Unsigned or stale signals = unmanaged.
TB2 → TB3 IdP to apps	SAML/OIDC assertions; SCIM provisioning	Per-app authentication policy tiers. Okta pushes users/groups to apps via SCIM so app access follows Okta group membership, not manual grants.
Inside TB3: 1Password	Vault keys	Separate cryptographic trust domain (Secret Key + account password). **Trade-off:** "Unlock with SSO" improves usability but collapses 1Password's independence into Okta. Recommendation: SSO unlock for staff, Secret Key + password retained for owners/admins.
Admin plane	Okta, GWS, Jamf, Falcon admin consoles	Separate admin identities, FIDO2 only, managed device required, no standing super-admin; break-glass accounts sealed in 1Password with alerting on use.

How posture feeds conditional access

Jamf answers *is this device managed?*, Falcon answers *is it healthy right now?*, and Okta combines both with the user's identity and the sensitivity of the app. Worked example for one engineer, Aisha:

Device	Gmail / Slack	1Password	AWS production console
Company MacBook (Jamf-managed, Falcon healthy)	**Allow** with FastPass	**Allow** with FastPass	**Limited:** role-gated (engineering), FIDO2 at every sign-in, 1-hour sessions, just-in-time elevation
Internet café PC (unmanaged)	**Block** by default. Browser-only, no-download access is possible only with GWS Context-Aware Access (Enterprise tier), so check licensing before promising it.	**Block.** A keylogger here would expose the master password and every secret behind it.	**Block**

Device trust gets a user through the door; the most sensitive systems still check again.

- **Known gap: posture is checked at sign-in, not continuously.** If Falcon detects malware ten minutes after Aisha signs in, her session stays valid until it expires. Continuous signal sharing (Falcon → Okta via Shared

Signals/CAEP, e.g. Okta Identity Threat Protection) is a paid add-on. Day one mitigation: a playbook on high/critical Falcon detections that contains the host and calls the same Okta `clear_sessions` and Slack notifier built in Part 1. Her next sign-in is denied automatically because the device now reports unhealthy. Medium detections go to an analyst instead, to limit false-positive disruption.

- **App sessions outlive the IdP session.** Clearing Okta sessions does not end the session AWS already issued at federation. AWS session duration is therefore kept short, and incident response includes revoking active sessions on the AWS role directly.

2. DLP and email security posture

At 50 people the real DLP risk is not a sophisticated insider; it is customer data shared to a personal Gmail, a public Drive link, an over-scoped OAuth app, or a secret pasted into Slack. Day one targets those paths with controls that have near-zero false positives. Content inspection, which is expensive to tune, comes later.

Channel	Day one (50 people)	Defer (to ~100–200)
Email (GWS)	Block automatic forwarding to external domains. Outbound content rule for card numbers (PAN with Luhn check), IBANs and Emirates ID patterns to external recipients: warn then quarantine. TLS enforced to known partners.	Gmail DLP with custom detectors and exact-data matching for customer records (Enterprise tier); attachment sandboxing beyond the Gmail default.
Drive / SaaS	External sharing limited to allowlisted partner domains; no "anyone with link" in shared drives that hold customer data. Third-party OAuth apps restricted to an allowlist (API controls). This is what makes Part 1's token revocation a short list rather than a surprise.	Drive DLP labels and auto-classification; SSPM/CASB for shadow SaaS once app count exceeds what one person can review quarterly; Slack DLP (Enterprise Grid only).
Endpoint	Jamf: FileVault, no local admin, removable storage read-only, AirDrop limited to managed contacts. Falcon USB device control in monitor mode. GitHub secret scanning with push protection.	Full endpoint content DLP (clipboard, print, upload inspection): highest false-positive rate and cost. Justified once there is a data-classification scheme people actually apply.
Policy	Three-tier data classification (Public / Internal / Confidential-Customer) written in one page; DLP rules map to the top tier only.	Labelling enforced in tooling.

SPF / DKIM / DMARC

These records stop outsiders *impersonating* Mal. They do nothing about real data leaving through real accounts: an employee mailing a spreadsheet to personal Gmail passes SPF, DKIM and DMARC. That is what the DLP controls above are for.

A fintech domain is a high-value spoofing target (fake payment instructions, fake support). The end state is **DMARC p=reject** on every domain, reached by a staged ramp, not on day one, because enforcing before every legitimate sender is aligned silently drops real mail such as transactional email, the HR system and invoices.

- **Day one:** inventory every sender. DKIM 2048-bit for GWS and every third-party sender, each signing as the Mal domain. SPF lists only real senders and stays under the 10-DNS-lookup limit. DMARC `p=none; rua=` to a report processor. Every parked or non-sending domain gets `v=spf1 -all`, DMARC p=reject and a null MX immediately, since there is nothing to break.
- **Weeks 2–6:** fix alignment from aggregate reports, then `p=quarantine` with pct ramping to 100, then `p=reject; sp=reject`. Move marketing/transactional mail to subdomains so a vendor compromise is isolated from the corporate domain.
- **SPF ~all vs -all:** with DMARC at reject, `~all` on sending domains is fine and avoids some receivers hard-failing forwarded mail before DMARC is evaluated; DMARC is the enforcement point. `-all` on non-sending domains.
- **Also day one:** MTA-STS (enforce) + TLS-RPT. **Deferred:** BIMI (needs a VMC/CMC; a brand feature, not a security control) and lookalike-domain monitoring (worth buying at around 100 staff).

3. What I cut in Part 1 and why

Run live: Okta (create user, add to `engineering`, clear sessions, suspend/deactivate) against an Okta Integrator org, and the Slack alert via a real incoming webhook, both on 31 October 2026, with screenshots and the Okta System Log in the README. **Mocked:** Google Workspace (the trial needs an owned domain and a delegated service account), 1Password (the SCIM Bridge must be deployed as a server) and Jamf (no free tenant). Mocks run through the *same* HTTP client as live calls, so payloads, retries and error paths are still exercised; 33 tests run at HTTP level. The Jamf XML payload is modelled on the Classic API `computerinvitations` resource and is unvalidated against a real instance.

Found by running it live, and fixed

What happened	Cause	Fix
Startup crash: Google Workspace switched to live mode	`.env.example` held example values; copied to `.env`, they looked like real credentials	Template values blanked (examples moved to comments); startup check with a clear error; test added
Okta returned HTTP 401 with a valid token	The org URL was still the template placeholder, so the token was sent to another tenant	Corrected the URL and rotated the token, since it had been sent somewhere it shouldn't. The 401 was handled correctly: not retried, dependent step skipped, failures reported.
Alert said "suspend" when Okta had deactivated the user	The label described the planned action; the user was PROVISIONED, and Okta only suspends ACTIVE users	The step now reports the actual outcome, e.g. `deactivated (was PROVISIONED)`; test updated

Known issues, not fixed

- **A rerun escalates.** A second leaver run finds the user already SUSPENDED and deactivates them. A retry should never do more than the first run; it should be a no-op.
- **No confirmation before a live leaver.** A typo in the email offboards the wrong person immediately.
- **Mocks are too forgiving.** The Okta fixture returns the same user for any email, so tests can't catch "acted on the wrong user".

Deviation from the brief: the brief suggests the 1Password Connect or Events API. Neither can change vault access: Connect manages items, and Events is read-only. Vault access in 1Password Business is granted to groups, and SCIM controls group membership and suspension, so I used SCIM. Events is the right tool for *verifying* a leaver afterwards, and I did not build that.

Shortcut in the build	Production at Mal
Custom script provisions GWS and 1Password directly.	**Biggest change:** Okta lifecycle management with SCIM push to GWS and 1Password, driven by HRIS as source of truth. The script shrinks to what Okta cannot do: GWS OAuth token revocation and signOut, the Jamf invite, cross-system alerting and evidence.
Okta SSWS API token (admin-scoped, long-lived).	OAuth 2.0 service app, `private_key_jwt`, scopes limited to `okta.users.manage` / `okta.groups.manage`.
GWS service-account JSON key on disk.	Workload identity federation with no long-lived key; delegation limited to the two scopes used.
Secrets from environment variables.	Pulled at runtime from 1Password Secrets Automation or a cloud secret manager; never in CI variables.
CLI trigger, in-process retries only.	HRIS webhook into a queue with idempotency keys and dead-lettering, so a failed step is retried and can't be lost when the process dies.
Audit log is a local JSONL file.	Shipped to the SIEM with immutable retention; the alert links to the run ID.
No read-back verification.	After the leaver flow: re-read Okta status, list GWS tokens (expect zero), check 1Password Events for later sign-in attempts; report fails if any check fails.
GWS account not suspended; no Jamf lock/wipe; no mover flow.	Suspend after manager handover of Drive/Calendar data; device lock on the termination date with remote wipe after the return window; mover = group diff with removals applied before additions.

4. Compliance mapping (ISO/IEC 27001:2022 Annex A)

A.5.18 Access rights — provisioning, review, modification and removal of access in line with policy.

How the build addresses it: the leaver flow removes access across IdP, email, OAuth grants and vaults in one run, with step-level timestamps and an explicit failure list. Each run writes a redacted record (run ID, subject, every action, result) to the audit log. That record is the removal evidence an auditor samples against the leaver list. Joiner access is assigned by group, not per user, which keeps grants consistent with a role.

Evidence, strongest first: (1) the **Okta System Log**, written by Okta rather than my script, with timestamp, actor, action and result for clear-session and suspend; (2) the run's audit record; (3) the Slack alert, which shows the team was told but is editable and short-lived, so it is a notification rather than evidence.

Gaps for audit: (1) there is no link to an authorising record. The leaver's email is typed on the command line, and Okta records the actor as the token's owner, so the log shows who *ran* it, not who *approved* it. In production the HR system triggers the flow and its ticket ID goes into the audit record. (2) There is no termination timestamp from HR, so the removal SLA (for example "same business day") can't be proven. (3) Periodic access reviews, which are part of this control, aren't covered at all. (4) Log integrity: a local file the operator can edit is weak evidence until it is shipped to immutable storage.

A.5.16 Identity management — the full lifecycle of identities is managed.

How the build addresses it: one identity (the work email) is created consistently across Okta, GWS and 1Password. Joiner calls are idempotent, so reruns don't create duplicates. Leaver suspends rather than deletes by default, which keeps the identity and its history attributable while access is removed. Deactivation is an explicit flag.

Gaps for audit: (1) there is no reconciliation. Accounts created outside the workflow (local SaaS logins, shared or service accounts) are invisible to it, and the auditor will ask how orphaned accounts are detected. A scheduled job diffing HRIS against Okta and each app closes this. (2) Service and non-human identities have no owner or review cycle. (3) There is no defined retention or deletion point after suspension.

Note: if Mal operates under a CBUAE licence, the same evidence supports the regulator's access-control expectations. I would check the specific technology-risk requirements against the licence category before claiming coverage.